SOFT COMPUTING PROJECT · COMPLETE GUIDE

FloodGuard AI

Reliability-aware sensor fusion with Interval Type-2 fuzzy logic for flood-risk detection: how it works, why it was built this way, what the results show, and how to explain it in a review or interview.

Code: github.com/usha-sree7/FlowGuard · Model version 0.4.0 · All numbers in this guide are measured from the project's own experiments.

Contents

1. The project in one minute
2. The problem it solves
3. The data
4. The pipeline, stage by stage
5. A complete worked example
6. How it was tested
7. Results, read honestly
8. How the code is organised
9. Running it and the live demo
10. Limitations and future work
11. Interview questions and answers
12. Glossary

---

1. The project in one minute

> **Say this:** "FloodGuard AI watches a river through its sensors (water level, how fast it is rising, and rainfall) and gives two separate answers every half hour. First, the flood risk: Normal, Watch, Warning or Critical. Second, how much to trust that answer: Confident, Uncertain or Insufficient data. Before deciding, it checks every sensor for faults like spikes, freezing or missing data, and gives faulty readings less weight. The decision itself is made by an Interval Type-2 fuzzy logic system with 64 expert rules, because the boundaries between 'high' and 'very high' water are themselves uncertain. I tested it on two real published river datasets by deliberately breaking the sensors. It clearly beats the baselines when sensors spike, its uncertainty index rises 2.4 to 6.1 times on faulty data, and it never claims to be confident when the water-level sensor is down."

---

2. The problem it solves

A typical flood alarm compares one water-level reading with one fixed threshold. That works only if the sensor is right. In real floods, sensors are exactly what fails:

- **Spikes.** Debris or electrical noise makes one reading jump. A threshold alarm fires falsely.
- **Stuck or missing readings.** A frozen or offline gauge hides a real rise, and the alarm keeps saying "Normal" with nothing behind it. This is the most dangerous error.
- **Disagreement.** The rain gauge and the radar disagree. With two sources you cannot tell which is wrong.

The research gap: classic systems output one crisp class and never say how trustworthy it is. FloodGuard keeps four ideas strictly separate:

Concept	Question it answers	Where it appears
Risk	How dangerous is the river?	Risk score 0–100 and class
Sensor reliability	Is this particular reading trustworthy?	Trust score 0–1 per sensor
Model uncertainty	How wide is the fuzzy system's answer?	Width of the score range (F)
Data sufficiency	Is the essential data even there?	Missing-data term (D)

The risk score is a fuzzy **severity index, not a probability** of flooding. The uncertainty index U is a **ranking index, not a calibrated probability** of error.

3. The data

	Bengalas River, Brazil (main)	Malwathu Oya, Sri Lanka
Source	Soares et al., TGIS 2026, UAE-ML repository	Rathnayake et al., PLOS ONE 2023
Size	72,011 readings, every 30 min, 2019–2023	4,765 daily readings
Sensors	Water level; rain gauge; weather radar (504 grid cells, averaged)	Water level (Kappachichiya); several rain gauges
Verification	SHA-256 checksum; radar cell count checked; target column = next level	SHA-256 checksum; 4,765 days rebuilt from lagged rows

Both datasets were downloaded from the authors' GitHub repositories (MIT licence). Schemas and counts were checked against the files themselves, and differences from the paper text are documented in `docs/PAPER_DATASET_AUDIT.md`. Soil moisture and river flow are not in either dataset, so they are not used.

> **Labels are proxies.** Neither dataset has official flood-event records. A time step is labelled by the class of the *observed water level one hour ahead* (one day for Malwathu), using thresholds of the training-period mean plus 1, 2 and 3 standard deviations. Mean + 2σ is the "flood threshold" rule the Bengalas authors used in their own code. The systems never see the future level they are scored against.

Radar bias: the radar reads about 2.7 times lower than the gauge, and its units are not documented. A single mean-field bias factor (total gauge rain ÷ total radar rain) is fitted on the training years only, which is standard hydrological practice.

4. The pipeline, stage by stage

1 Sensors	2 Reliability	3 Fusion	4 IT2 fuzzy	5 Outputs	6 Dashboard
level, rise, rain	trust 0–1	estimate + interval	64 rules, KM	risk + U	replay, alarms

Everything is **causal**: at each step only past and current readings are used, so the same code could run on a live feed.

Stage 1: Inputs

- **Water level** in metres.
- **Rise rate** in metres per hour: (level now − previous level) ÷ time step. Computed only across one normal step; after a gap it is left empty rather than guessed.
- **Rainfall** accumulated over a short window: the last 2 hours for Bengalas, the last 2 days for Malwathu.

Stage 2: Sensor reliability

Every sensor gets a trust score between 0 and 1 at every step. It starts at 1, and each check that fires lowers it. A score below 0.5 counts as "flagged".

Check	How it works
Missing / out of range	No reading, or a value outside the physically possible range → trust 0.
Spike	The reading jumps away from the median of recent readings by more than 1.5 × the 99.9th-percentile jump seen in training, and rainfall does not explain it.
Stuck	The level stays exactly flat for at least 3 hours while it is raining heavily (a real river would respond).
Noise	The roughness of the signal (second differences) is far above its normal training level.
Rain inconsistency	With 3+ gauges: the odd one out is down-weighted. With only 2 sources: you cannot tell which is wrong, so both get a mild penalty and the rest is recorded as **unresolved conflict**, which widens the uncertainty instead.
Device health	Maintenance or battery metadata, if supplied, lowers the prior trust.

All limits are learned from the training years only.

Stage 3: Reliability-weighted fusion

$$c = r \cdot x + (1 - r) \cdot p$$

Here x is the new reading, r its trust score, and p the median of the last 4 trusted readings. With r = 1 the reading is used as it is. With r near 0 the estimate falls back on recent good data, so one bad value cannot trigger an alarm.

Fusion also builds an **interval** around each estimate. The less the trust, the longer the sensor has been missing, or the more two rain sources disagree, the wider the interval. Rain is a trust-weighted mean of the bias-corrected gauge and radar.

> **Key design decision (version 0.4):** the risk decision uses the fused *point* estimates; the intervals feed only the uncertainty output. Version 0.3 decided risk from the intervals, which turned uncertainty into higher risk: with 30% missing data its F1 fell to 0.281. That run is kept as a documented negative result.

Stage 4a: Fuzzification (numbers → words)

Each input is described by four linguistic terms with soft trapezoid edges. Approximate Bengalas values:

Input	Terms
Water level	Low (below ~1.1 m) · Elevated (~1.2–1.4 m) · High (~1.5–1.65 m) · VeryHigh (above ~1.8 m)
Rise rate	Falling · Steady (~−0.06 to +0.10 m/h) · Rising (~0.26–0.69 m/h) · RisingFast (above ~1.0 m/h)
Rain (2 h)	None (below ~1) · Light (~1–5) · Moderate (~7–13) · Heavy (above ~18)

The boundaries are not hand-picked: they are anchored on statistics of the river's own training data (for example mean + kσ of the level). Because the edges are soft, 1.45 m can be partly Elevated and partly High at once.

What makes it Type-2. Each boundary is itself uncertain: computed separately on each calendar year, the flood threshold moves. The standard error of each anchor across years (about 0.12 m for the Bengalas flood threshold) sets the gap between a **lower** and an **upper** membership curve. The band between them is the **footprint of uncertainty (FOU)**. So each input gets a membership *interval*, not a single number. Type-1 fuzzy logic is the special case where the two curves coincide.

Stage 4b: The 64-rule base

4 terms × 4 terms × 4 terms = 64 combinations, and every one has a rule, so no situation is undefined. The rules are expert-written (there are no real flood labels to learn from) and follow one template:

1. The **level sets the base class**: Low → Normal, Elevated → Watch, High → Warning, VeryHigh → Critical.
2. **+1 class** if the level is rising fast, or rising with moderate or heavy rain.
3. **+1 more** with heavy rain, a rising level, and a level already above Low.
4. **−1 class** if the river is Elevated or High but steady or falling, with no rain (likely receding).
5. Heavy rain always gives **at least Watch**, even at a low level (early warning).

Examples: *IF level is High AND rise is Rising AND rain is Moderate THEN Critical. IF level is Low AND rise is Steady AND rain is None THEN Normal.*

"AND" is the product: the memberships are multiplied. Because memberships are intervals, each rule gets a **firing interval** [lower, upper].

Stage 4c: Type reduction and the risk class

Each class has a target score interval. The **Karnik–Mendel (KM)** algorithm finds the lowest and highest weighted average of these targets that the firing intervals allow. That gives a score range $[y_l, y_r]$. Its

midpoint is the risk score.

Class	Rule target	Score band → class
Normal	5–15	below 25
Watch	32–43	25 to 50
Warning	57–68	50 to 75
Critical	85–95	75 and above

The project implements both the classical iterative KM algorithm and an exact vectorised version that tests every switch point. A unit test confirms they agree to within 10^{-9} on 1,500 random cases.

Stage 5: The uncertainty index and status

$$U = 100 \cdot (0.4 \cdot Q + 0.3 \cdot D + 0.3 \cdot F)$$

Part	Meaning
Q · quality deficit	How much trust the sensors have lost right now (from Stage 2)
D · data deficit	Which essential readings are missing; a missing water level counts most
F · model uncertainty	Width of the KM score range, scaled against typical clean-data widths

Status	Rule
Confident	No missing data (D = 0) and U below 25
Insufficient data	Water level missing for more than 2 steps, or a large data deficit
Uncertain	Everything else

Safety rule: the status can never be “Confident” while the water level is missing. This is tested, and counted in every experiment: zero occurrences. The weights 0.4 / 0.3 / 0.3 are provisional.

Stage 6: The dashboard

A web app replays the held-out test period step by step, showing risk, the score range, U, the status and each sensor’s trust. It is labelled as a replay; no live feed is connected.

5. A complete worked example

Inputs typed into the running app (IT2 Model page → Try the inference): **level 1.75 m, rise 0.8 m/h, rain 15**.

1. **Fuzzify.** 1.75 m is mostly VeryHigh and partly High. 0.8 m/h is Rising, slightly RisingFast. 15 is between Moderate and Heavy.
2. **Fire rules.** The strongest are “VeryHigh + Rising + Moderate → Critical” (firing interval 0.04–0.70), “VeryHigh + Rising + Heavy → Critical” and “High + Rising + Moderate → Critical”.
3. **Type-reduce.** Almost all the weight is on Critical rules, so KM gives the range 85–95.

4. **Classify.** The midpoint is 90, which is 75 or above → **Critical**.
5. **Uncertainty.** In the live pipeline, if all sensors were clean and complete, Q and D would be 0 and only the range width F would contribute to U; with U under 25 the status would be Confident. If the level sensor had just spiked, its trust would drop, the estimate would lean on recent readings, and U would rise.

6. How it was tested

- **No leakage.** The data is split by time: the first 70% is training, the last 30% is testing, never shuffled. Every parameter (thresholds, membership anchors, FOU, radar bias, reliability limits, F scaling) is fitted on clean training data only. A unit test confirms that tampering with test data leaves the calibration unchanged.
- **Fault injection.** Faults are injected into a *copy* of the test data in 11 scenarios: clean; low, medium and high noise; 10% and 30% missing; spikes; stuck; rain-source conflict; reduced reliability; and all combined.
- **Ground truth.** Labels come from the original, unbroken data. The fault masks are used only for scoring, never by the model.
- **Four systems on identical input:** (1) a threshold on the water level; (2) a Type-1 fuzzy system; (3) the IT2 fuzzy system with no reliability layer; (4) the proposed system (reliability + fusion + IT2 + U).
- **Metrics:** alert F1 for "Warning or higher", precision, recall, false-positive rate, a severity-weighted error that counts under-warning twice as heavily, fault-detection rates, and U on faulted vs unfaulted steps.
- **Seeds.** Fault placement is random, so experiments are repeated with 5 seeds and means are reported.

Each run writes `results/runs/<id>/` : results.json with full metadata (seed, checksums, configuration, calibration, rule base, model version), summary.csv, the corrupted inputs, and the ground-truth masks.

7. Results, read honestly

Bengalas, alert F1, mean of 5 seeds (bold = best in the row):

Scenario	Threshold	Type-1	IT2	Proposed
Clean data	0.862	0.883	**0.895**	0.888
Spikes	0.748	0.760	0.768	**0.835**
Missing 30%	0.836	0.839	**0.845**	0.825
Heavy noise	**0.721**	0.642	0.645	0.636
Reduced reliability	0.851	0.865	**0.875**	0.818
All combined	0.662	0.652	0.654	**0.667**

- **IT2 vs Type-1:** with identical rules, IT2 is slightly better on clean data (0.895 vs 0.883). The gain is small, and that should be said.
- **Clear win: spikes.** 0.835 ± 0.011 vs 0.768 ± 0.014 for the best baseline: about 6.7 points, far beyond the seed variation. This is what the reliability layer is designed for.
- **Combined faults:** nominally best, but within seed-to-seed variation, so call it a tie. (A single-seed run on 26 Oct 2026 gave 0.639 vs 0.651 for the threshold.)
- **Worse:** heavy noise (the plain threshold wins, because the rise rate amplifies noise and the fused estimate lags genuine rises), missing data, reduced reliability, stuck sensors, and every scenario on the

daily Malwathu data.

- **Uncertainty works as intended:** U is 2.4–6.1× higher on faulted steps than unfaulted ones, and "Confident while level missing" = 0 in every run.
- **Fault detection (level sensor):** spikes recall 0.93 but precision about 0.14 (many false flags); missing 1.00; stuck only 0.01, a known weakness.
- **Speed:** about 0.01–0.03 ms per observation for the full pipeline; a full 11-scenario Bengalas experiment takes about 25 s.

8. How the code is organised

File	What it does
`datasets/registry.py`	Verified metadata: URLs, SHA-256 checksums, column mapping, label info, limitations
`datasets/loaders.py`	Download, checksum, read the real schema, derive the rise rate, quality report
`calibration.py`	Chronological split; all parameters from the training split only
`reliability.py`	Per-sensor checks and trust scores; unresolved conflict
`fusion.py`	Trust-weighted point estimates and intervals
`fuzzy/mf.py`	IT2 trapezoids (lower/upper curves), exact interval firing
`fuzzy/km.py`	Karnik–Mendel type reduction (iterative and exact)
`fuzzy/fis.py`	The 64 rules, IT2 and Type-1 inference
`uncertainty.py`	Q, D, F → U, bands and status
`systems.py`	The four compared systems; proxy labels
`faults.py`, `metrics.py`, `experiment.py`	Fault injection, scoring, experiment runs and storage
`api.py`	FastAPI web server; also serves the React frontend
`frontend/src/pages/`	Dashboard, Dataset Explorer, IT2 Model, Experiments

All paths under `backend/floodguard/` unless shown otherwise. Stack: Python 3.13, NumPy, pandas, FastAPI, uvicorn; React + TypeScript built with Vite. 25 automated tests (pytest), all passing.

9. Running it and the live demo

```
cd backend
pip install -r requirements.txt
python scripts/import_dataset.py bengalas
python scripts/import_dataset.py malwathu
cd ../frontend && npm install && npm run build
```

```
cd ../backend
python -m uvicorn floodguard.api:app --port 8000
# open http://127.0.0.1:8000
```

Tests: `python -m pytest -q`. Experiment: `python scripts/run_experiment.py bengalas` (about 25 s).

Demo script (about 3 minutes)

1. **Dataset Explorer:** show the source paper, checksums, row counts and quality report. Point out that the labels are proxies.
2. **IT2 Model:** show a membership function with its lower and upper curves (the FOU). Type level 1.75, rise 0.8, rain 15 and show the Critical result and the rules that fired. Lower the level to 1.2 and watch the class fall.
3. **Dashboard:** choose scenario “spikes”, click *Next high-water event*, then *Play*. Show the level sensor’s trust dropping on a spike, U rising, and the comparison panel with all four systems next to the proxy label.
4. **Experiments:** show the F1 table, U faulted vs unfaulted, and “Confident while level missing = 0”.

10. Limitations and future work

Limitations: proxy labels only; one water-level sensor per site; no advantage yet under noise or missing data; stuck faults rarely detected; no benefit on daily data; U weights and thresholds provisional; the v0.4 design was chosen after seeing test results, so it is not a fully blind evaluation.

Next steps: a trend-aware (Kalman-style) fused level estimate; a better stuck detector; a separate validation block for all tuning; event-level and lead-time metrics; PSO tuning of membership functions and U weights on validation data; an ANFIS baseline; one more catchment with recorded flood events.

11. Interview questions and answers

What is the main soft-computing contribution?

A reliability layer and evidence fusion feeding a genuine Interval Type-2 fuzzy system (FOU, interval firing, 64 rules, Karnik–Mendel type reduction), which outputs flood risk and, separately, an uncertainty/data-sufficiency status.

Why fuzzy logic at all, and not a simple threshold?

Flood danger depends on several things at once (how high, how fast, how much rain) and the boundaries between categories are gradual. Fuzzy rules capture this the way a hydrologist would explain it, and every decision can be traced to the rules that fired.

Why Type-2 rather than Type-1?

Because the class boundaries are themselves uncertain estimates; they shift from year to year. Type-2 models that uncertainty directly through the FOU, and the width of its output range gives a principled model-uncertainty signal. Measured with identical rules, IT2 scored 0.895 vs 0.883 for Type-1 on clean Bengalas data and 0.864 vs 0.855 on Malwathu. The gain is small.

Where does the FOU come from? Did you just pick it?

No. Each half-width is the standard error of that anchor across calendar years of training data. For Bengalas, the mean + 2σ threshold has a standard error of 0.121 m.

Is the type reduction real Karnik–Mendel?

Yes. The classical iterative algorithm is implemented, plus an exact version that checks every switch point. Tests confirm they agree to 10^{-9} on 1,500 random cases.

Why not ANFIS or deep learning?

There are no observed flood labels, only proxies built from the water level, so a trained model would just learn the proxy rule. Flood events are rare (653 positive test steps), and the rule base is explainable. ANFIS, ML and PSO were not run, so no claim of superiority over them is made; they are listed as future work.

Aren't you validating on your own rules?

The labels come from the *future* observed level in the original data, using training-period thresholds. Every system predicts something it does not see. The thresholds (mean + kσ) match the dataset authors' own rule. They are still proxies, not observed flood impacts, and this is stated.

How did you prevent data leakage?

A chronological 70/30 split; every parameter fitted on clean training data only; faults injected into copies; labels from the original test data; fault masks used only for evaluation; and a unit test that checks the calibration does not change when test data is altered.

Does your system beat the baselines?

Only partially. A clear win under spikes (0.835 vs 0.768). A tie under combined faults. Slightly worse on clean data, under noise, missing data, stuck readings and reduced reliability, and on the daily dataset. Its main added value is the uncertainty output, which reliably flags degraded data.

Why is it worse under noise?

Fusion pulls noisy readings towards recent trusted values, which lags genuine rises (recall drops from 0.827 to 0.640 under heavy noise). All fuzzy variants also use the rise rate, and differencing amplifies noise, which is why the plain threshold wins there.

You changed the design after seeing results. Isn't that cheating?

It is disclosed. Version 0.3 converted uncertainty into higher risk (F1 0.281 at 30% missing). Version 0.4 separates risk from uncertainty. Because the change came after seeing test results, the v0.4 numbers are not fully blind; the next protocol adds a validation block.

Is U a probability? Is the risk score a probability?

Neither. $U = 100 \cdot (0.4Q + 0.3D + 0.3F)$ is a ranking index with provisional weights. The risk score is a fuzzy severity index.

Can it say "Confident" when the level sensor is down?

No. Confident requires $D = 0$ and $U < 25$; a missing level makes $D > 0$, and after 2 steps the status becomes Insufficient data. It is tested, and counted in every experiment: zero occurrences.

How do you detect faulty sensors?

Missing or out-of-range values; spikes (a jump from the recent median, unless rain explains it); stuck readings (flat level during heavy rain); noise (roughness of second differences); rain-gauge inconsistency; device-health metadata. Spike recall is 0.93 with precision 0.14; stuck recall is only 0.01, a known weakness; missing is 1.00.

What if the two rain sources disagree?

With only two sources you cannot tell which is wrong, so neither is arbitrarily blamed. The disagreement is recorded as unresolved conflict that widens the intervals and raises U. With three or more gauges (Malwathu), the outlier is down-weighted.

Are the datasets real? How did you verify them?

Yes: downloaded from the authors' GitHub repositories and pinned by SHA-256. Bengalas has 72,011 rows; the radar cell count (504) and the target column (equal to the next level) were verified. Malwathu's 4,765 days were reconstructed from its lag-structured rows. Discrepancies, such as an unconfirmed ">73,000 tuples" claim and unclear rain units, are documented.

Why are soil moisture and river flow not used?

Neither dataset contains them, and the project does not invent variables.

How fast is it? Is the dashboard live?

About 0.01–0.03 ms per observation. The dashboard is a replay of the held-out test period, labelled as such. The pipeline is causal, so it could run on a live feed, but none is connected.

12. Glossary

Term	Meaning
Membership function	A curve giving how strongly a value belongs to a word, from 0 to 1
Interval Type-2 (IT2)	Fuzzy sets whose membership is an interval between a lower and upper curve
FOU	Footprint of uncertainty: the band between the lower and upper curves
Firing strength	How strongly a rule applies; in IT2 it is an interval
Karnik–Mendel (KM)	Algorithm that turns the interval rule outputs into a score range $[y_l, y_r]$
Type reduction	Turning a Type-2 result into a Type-1 interval before defuzzifying
Defuzzification	Turning the fuzzy result into one number (here, the midpoint of the KM range)
Reliability / trust score	0–1 weight saying how believable a sensor reading is right now
Fusion	Combining readings, weighted by trust, into one estimate plus an interval
Proxy label	A stand-in for a true flood label: the future level crossing a statistical threshold
Chronological split	Train on earlier time, test on later time, so no future information leaks
F1 score	Balance of precision (few false alarms) and recall (few missed floods)
Causal	Uses only past and current data at each step